

Repository Intelligence & Knowledge Graph — Lifecycle Audit

Date: 2026-06-30
Scope: `levelup-ai-qa-agent` (backend) Repository Intelligence pipeline
Author: Engineering audit (pre-merge review, PR #215 follow-up phase)
Goal: Trace the full lifecycle of the “Knowledge Graph”, document construction / storage / consumers, identify gaps, and recommend a **Knowledge-Graph-First** architecture that moves all AI features off repeated repo scans and oversized prompt summaries.

1. Executive Summary

There is **no single knowledge graph** today. Repository Intelligence is fragmented into **three independent representations** that are built separately, stored in different places, and never linked to one another:

#	Representation	Built by	Stored in	Shape
A	RepositoryProfile (flat profile)	<code>context/repository-context-engine.ts</code> (+ <code>ast-analyzer.ts</code>)	<code>repository_contexts.profile</code> (JSONB)	Flat arrays of helpers, page objects, fixtures, flows, coding style
B	Method Intelligence index + call graph	<code>services/method-intelligence-service.ts</code> (ts-morph)	<code>repository_methods</code> (nodes) + <code>method_dependencies</code> (edges) — relational	Nodes + edges (the only real graph)
C	Code chunks + vector embeddings (RAG)	<code>services/code-chunk-embedder.ts</code> + <code>embedding-service.ts</code>	<code>code_chunks</code> + pgvector	Embedded text chunks for semantic retrieval

The component literally named the **Knowledge Graph** — `services/knowledge-graph-service.ts` — is a read-only projection of representation B. It materializes `repository_methods` + `method_dependencies` into a `{ nodes, edges }` JSON document on demand. **It has exactly one consumer: a D3 visualization API route.** No generative AI feature (Test Case Lab, Script Generation, Healing, RCA, AI Review) queries it.

The single most important finding: the call graph (`method_dependencies`) is the highest-value asset in the system and is almost entirely unused for reasoning. Every AI feature instead either (a) fuzzy-matches method names (no relationships), or (b) stuffs the flat `RepositoryProfile` JSON into the prompt. The graph edges are queried in only two places — the D3 viz endpoint and Impact Analysis — and Impact Analysis bypasses the graph service with its own bespoke recursive SQL.

2. How the Graph Is Constructed

2.1 Ingestion orchestration

The entry point is `services/repo-scan-service.ts` → `scanAndPersistRepo()` , invoked from:

- the synchronous HTTP scan route, and
- `jobs/workers/repo-analysis-worker.ts` (async `scan` / `rescan` / `embed` jobs).

A single scan runs **three passes in sequence**, each independently feature-flag gated:

```
clone/checkout repo
├── Pass 1: RepositoryContextEngine.scan(repoRoot)
│   └── → RepositoryProfile + CodeChunk[] → persist profile JSONB +
code_chunks rows
├── Pass 2 (if VECTOR_SEARCH): code-chunk-embedder → embeddings into pgvector
└── Pass 3 (if METHOD_INTELLIGENCE): MethodIntelligenceService.analyzeRepository(repoRoot, contextId)
    └── → repository_methods (nodes) + method_dependencies (edges)
```

The three passes share the same `repository_context_id` but produce **disconnected data** — there is no foreign key from a profile `FunctionSignature` to a `repository_methods.id` , and `code_chunks` are not linked to method nodes. They are three views that never reference each other.

2.2 Node construction (`repository_methods`)

`MethodIntelligenceService.analyzeRepository()` (`services/method-intelligence-service.ts`):

1. Walks the repo (`discoverFiles` , skips `node_modules` / `dist` /etc., caps 2000 files / 500 KB each).
2. For each `.ts/.tsx/.js/.jsx/.mjs/.cjs` file, uses **ts-morph** to extract:
 - standalone function declarations,
 - exported const arrow / function expressions,
 - class methods (page objects, helper classes).
3. For each method it records: name, file path, class name, parameters (name+type), return type, `isAsync` , **full source code** (≤ 20 KB), `codeHash` (SHA-256 of whitespace-normalized source, for dedup), line range, JSDoc description, and a heuristic `methodType` .

Node types (`method_type` column, set by `classifyMethod`):

- `test` — file matches `*.spec/.test` or name looks test-y
- `page_object_method` — class matches `/page|screen|view|component/i` or path under `pages/`
- `utility` — path under `utils|helpers|lib|common|support|fixtures`
- `helper` — everything else (the catch-all)

Persisted via `replaceRepositoryMethods(contextId, records)` — a **full snapshot replace** per scan (no incremental/diff update).

2.3 Edge construction (`method_dependencies`)

While extracting each method, `extractCalledMethods()` walks the AST body and collects the **bare last-segment name** of every call expression (`foo.bar.baz()` → `"baz"` ; `helper()` → `"helper"`).

After all nodes are persisted, the service builds a **name→id map** and, for each caller, looks up each callee name in that map. An edge (`caller_method_id`, `callee_method_id`, `call_count`) is written via `upsertMethodDependency` **only when the callee name resolves to an indexed method**. External/library calls (`expect` , `page.click` , `npm imports`) are dropped.

Edge limitations (important):

- Edges are resolved by **bare name only** — no scope/import/type resolution. Two different methods named `login` collapse; an overloaded/duplicated name produces ambiguous edges.
- Only **calls** relationships exist. There are no `imports` , `extends` , `implements` , `uses-fixture` , `asserts-with` , `waits-with` , `reads-test-data` , or `tests→page-object` semantic edges.
- No edge to external symbols, so framework/synchronization usage is invisible at the graph level.

2.4 Graph materialization (`KnowledgeGraphService`)

`services/knowledge-graph-service.ts` is a **pure-PostgreSQL, on-demand projection** (explicitly no Neo4j, documented in its header). `buildGraph(repoContextId)` :

- emits **method nodes** (`m:<id>` , `kind method` or `test`),
- emits **file nodes** (`f:<path>` , `kind file`),
- emits `in_file` edges (`file→method`),
- emits `calls` edges (`method→method`) and re-labels them `tests` when the caller is a test method,
- `getMethodNeighborhood(methodId, depth)` runs a recursive CTE to expand callers+callees up to `depth` hops,
- `exportForD3()` converts to `{ nodes, links }` .

Graph node kinds: `method` | `file` | `test` . Graph edge types: `calls` | `in_file` | `tests` .

3. Where It Is Stored

Data	Store	Persistence	Notes
RepositoryProfile	<code>repository_contexts.profile</code> JSONB (Postgres)	Durable, one row per repo/company/project	The blob actually fed into prompts
Method nodes	<code>repository_methods</code> table	Durable, full-replace per scan	Includes full source text + <code>code_hash</code>
Call edges	<code>method_dependencies</code> table	Durable	B-tree indexes on caller/callee; <code>pg_trgm</code> GIN index on <code>method_name</code> for fuzzy search
Materialized graph {nodes, edges}	Nowhere — computed on demand	Ephemeral	Rebuilt per API request; no cache
Code chunks	<code>code_chunks</code> + pgvector embeddings	Durable (if RAG enabled)	Separate semantic store
Health snapshots / quality issues	<code>repository_health_snapshots</code> , <code>code_quality_issues</code>	Durable	Phase 3C, also keyed by <code>method_id</code>

There is no graph database and no persisted graph document or cache. The graph is recomputed from relational rows on every request. Feature gating: `METHOD_INTELLIGENCE` (must be on for nodes/edges to exist) → `KNOWLEDGE_GRAPH` (projection) → `RAG_ENABLED` + `VECTOR_SEARCH` (embeddings) → `HEALTH_INTELLIGENCE`. All default **off**.

4. Which Features Query It (and What They Retrieve)

Feature	Source file	What it reads	Uses graph edges?	Uses method index?	Uses flat profile?
Knowledge-Graph viz	api/routes/ repo-intel- li- gence-3c.ts	buildGraph / getMethod- Neighborhood → D3	✓ (only true consumer)	✓	—
Impact Analysis	services/im- pact-analys- is-ser- vice.ts	Own recurs- ive CTEs over meth- od_dependenc ies (ana- lyzeMeth- odImpact , findBreak- ingTests , analyze- FileImpact)	✓ (bypasses graph ser- vice)	✓	—
Script Gen- eration	script-gen/ script-gen- engine.ts , engines/ test-to- script-en- gine.ts	TrueReuseEn- gine.findExi- stingHelper/ buil- dReuseContex- t → search- Methods (fuzzy name match); In- telligence- FusionSer- vice → pro- file summary; repo-pat- tern-analyz- er + prompt- builder → profile JSON; framework- auditor → impact-ana- lysis	⚠ only indir- ectly via framework- auditor	✓ (name only)	✓
Healing	services/ healing-in-	searchMeth- ods (name)	✗	✓ (name only)	✗

Feature	Source file	What it reads	Uses graph edges?	Uses method index?	Uses flat profile?
	telligence-context.ts	+ RAG find-SimilarCode			
Test Case Lab	engines/test-coverage-engine.ts	Flat RepositoryIntelligenceSummary (summary, techStack, patterns, architecture) injected as prose	✗	✗	⚠ shallow summary only
RCA Engine	engines/rca-engine.ts	— none —	✗	✗	✗
AI Review	script-gen/ai-review-engine.ts	— none —	✗	✗	✗

What each retrieval looks like

- **Script Gen / Healing reuse** = `searchMethods(contextId, term, {methodType, limit})` — a `pg_trgm / ILIKE` **fuzzy name lookup**. It returns candidate methods by name similarity; it does **not** traverse who-calls-whom, so it cannot answer “what helper do existing tests use to log in?” via relationships — only via name guessing.
- **Impact Analysis** = recursive CTE walking `method_dependencies` upward (callers) to find affected tests, downward (callees) for call chains. This is the one feature that genuinely reasons over relationships — but it reimplements traversal instead of using `KnowledgeGraphService`.
- **Test Case Lab** = receives a hand-rolled `RepositoryIntelligence` object (summary string + tech-stack/pattern arrays) and pastes it into the prompt as prose. It is disconnected from both the method index and the graph.
- **IntelligenceFusion** = reads `repository_contexts.profile`, then `summarizeRepoProfile()` reduces it to **counts** (`helpersCount`, `pageObjectsCount`, `fixturesCount`) plus framework/language — so even the rich profile is flattened to scalars before reaching the prompt.

5. Where the Graph Is NOT Used But Should Be

1. **Script Generation reuse is name-based, not relationship-based.** The generator asks “is there a method named like `login` ?” instead of “which page-object method do the existing login tests actually call, and what assertion/wait/test-data helpers travel with it?” The `tests → calls → method` edges already encode the answer and are ignored. This is the direct root of the class of bugs PR #215 is fixing (e.g. emitting `loginPage.login('username', 'in')` instead of reusing the real flow).

2. **Test Case Lab gets a shallow prose summary.** It never sees real page objects, business flows, or which methods exist — so generated test cases cannot reference real capabilities or coverage gaps. The graph + profile could feed it concrete “these flows/methods exist, these are untested” facts.
3. **RCA and AI Review consume zero repository intelligence.** RCA classifies failures with no knowledge of the call graph (what the failing test calls, what changed upstream). AI Review critiques generated scripts without knowing the project’s real helpers/conventions — so it cannot flag “you reimplemented `LoginPage.login()`” or “you used a hard sleep where the repo uses web-first assertions.”
4. **Impact Analysis duplicates traversal logic.** It should be a consumer of one graph traversal API, not a parallel implementation. Today there are two recursive-CTE code paths over the same edges.
5. **The three representations are not linked.** Profile helpers, `repository_methods` nodes, and `code_chunks` cannot be joined. A query like “give me the source + embeddings + callers of `LoginPage.login`” requires three disjoint lookups with no shared key. Reuse, RAG, and graph evidence cannot corroborate each other.
6. **No semantic edges.** The graph only has `calls`. It lacks `imports`, `extends/implements`, `uses-fixture`, `reads-test-data`, `asserts-with`, `synchronizes-with`, and `test → exercises → flow`. These are exactly the relationships that make a generated test indistinguishable from a senior engineer’s — and they are precisely the “method semantics, synchronization strategies, relationships between objects” the next phase calls for.
7. **No persistence/caching of the materialized graph and full-replace ingestion** — every consumer pays a rebuild cost, and there is no incremental update on file change, which blocks cheap, frequent graph refresh.

6. Recommended “Knowledge-Graph-First” Architecture

Principle: every AI feature should answer questions by **querying one Repository Knowledge Graph (RKG)** — never by re-scanning the repo or pasting a large profile blob into a prompt. Prompts receive small, query-scoped, relationship-grounded facts retrieved from the graph.

6.1 Unify the three representations behind one graph

Promote `repository_methods` / `method_dependencies` into the canonical RKG and **attach** the other two representations to it:

- Link profile assets (helpers, page objects, fixtures, flows, data files) to their `repository_methods.id` node (add a node `role` / `category` and the profile metadata onto the node, or a `node_attributes` JSONB).
- Link `code_chunks` (and their embeddings) to the owning method/file node via a `method_id` / `node_id` FK — so retrieval can return node + source + embedding + neighbors in one hop. This fuses graph traversal with RAG (GraphRAG).

6.2 Enrich the schema with semantic node & edge types

- **Node types:** `method`, `test`, `page_object`, `fixture`, `helper`, `utility`, `file`, `flow`, `data_file`, `selector/locator`, `env_config`.

- **Edge types:** keep `calls / in_file / tests`; add `imports`, `extends`, `implements`, `uses_fixture`, `reads_test_data`, `asserts_with`, `synchronizes_with` (carry the `WaitStyle`), `logs_with` (carry the `LoggingStyle`), `exercises_flow`, `locates` (method→selector). Resolve `calls` edges with **import/scope awareness** (not bare name) to remove the ambiguity noted in §2.3.
- Attach **method semantics** to nodes: classification (already have `methodType`), a short capability summary, the assertion/wait/logging style observed, and a confidence.

6.3 One Graph Query API (the contract every feature uses)

Expose a single service (extend `KnowledgeGraphService`) with intent-level queries, e.g.:

- `getReuseCandidatesForIntent(contextId, intent)` → page-object method + the assertion/wait/data helpers it co-occurs with (graph neighborhood, not name match). **Script Gen + Healing** call this.
- `getNeighborhood(nodeId, depth, edgeTypes[])` → typed subgraph. **Impact Analysis** is refactored to call this instead of its own CTEs.
- `getFlowsAndCoverage(contextId)` → flows, the methods/tests that implement them, untested gaps. **Test Case Lab** calls this.
- `getChangeImpact(nodeId)` → affected tests + call chains. **RCA / Impact / AI Review** call this.
- `graphRagRetrieve(contextId, query)` → vector search over chunks **re-ranked by graph proximity**, returning node + source + neighbors.

All return **compact, typed evidence objects** sized for prompt injection — replacing today’s “summarize the whole profile into the prompt” approach.

6.4 Make every feature a graph consumer

- **Script Generation:** before generating any Playwright code, query the graph for the reuse subgraph of the test’s intent and prefer existing nodes; only synthesize when the graph returns nothing. This directly advances the 8.8 → 10 target.
- **AI Review:** validate generated scripts against the graph — flag reimplementations of existing nodes, convention/synchronization mismatches (`synchronizes_with` / `logs_with` edges), and unused-but-available helpers.
- **RCA:** when a test fails, pull its `calls / exercises_flow` neighborhood + recent upstream changes to ground root-cause hypotheses.
- **Test Case Lab:** generate cases from real flows + coverage gaps, not a prose summary.

6.5 Storage, caching & incremental updates

- Keep Postgres as the store (no Neo4j needed at current scale); add a **materialized, versioned graph document** (per `repository_context_id`, keyed by a content hash) cached so consumers don’t rebuild per request. Invalidate on rescan.
- Move ingestion from **full-replace** to **incremental** (diff changed files → upsert affected nodes/edges) so the graph can refresh cheaply and frequently.
- Add a graph-completeness/health metric (nodes, edges, unresolved-callee ratio, link coverage across the three representations) surfaced via `intelligence-health-service`.

6.6 Phasing

1. **Link layer (highest ROI, low risk):** add FKs joining profile assets + `code_chunks` to `repository_methods`; add the unified Graph Query API as a thin wrapper over existing tables. No behavior change, unlocks everything else.
2. **Semantic edges:** import/scope-aware `calls`, plus `uses_fixture` / `reads_test_data` / `asserts_with` / `synchronizes_with`.

3. **Feature migration:** point Script Gen reuse, then AI Review, RCA, Test Case Lab at the Graph Query API; refactor Impact Analysis onto it.
4. **GraphRAG + caching + incremental ingestion.**

7. Appendix — Key Files

Concern	File
Profile build (AST)	<code>src/context/repository-context-engine.ts</code> , <code>src/context/ast-analyzer.ts</code> , <code>src/context/types.ts</code>
Method index + call-edge build	<code>src/services/method-intelligence-service.ts</code>
Graph schema (tables/indexes)	<code>src/db/postgres.ts</code> (<code>migrateMethodIntelligence</code>)
Graph projection (nodes/edges/D3)	<code>src/services/knowledge-graph-service.ts</code>
Impact traversal (parallel CTEs)	<code>src/services/impact-analysis-service.ts</code>
Reuse (name-based)	<code>src/services/true-reuse-engine.ts</code>
Prompt fusion (profile→counts)	<code>src/services/intelligence-fusion-service.ts</code>
RAG / embeddings	<code>src/services/rag-service.ts</code> , <code>src/services/code-chunk-embedder.ts</code> , <code>src/services/embedding-service.ts</code>
Ingestion orchestration	<code>src/services/repo-scan-service.ts</code> , <code>src/jobs/workers/repo-analysis-worker.ts</code>
Consumers — Script Gen	<code>src/script-gen/script-gen-engine.ts</code> , <code>src/engines/test-to-script-engine.ts</code>
Consumer — Healing	<code>src/services/healing-intelligence-context.ts</code>
Consumer — Test Case Lab	<code>src/engines/test-coverage-engine.ts</code>
Consumer — viz/impact API	<code>src/api/routes/repo-intelligence-3c.ts</code>
Feature flags	<code>src/config/features.ts</code> (<code>REPO_INTELLIGENCE.*</code>)